Learning Objectives

Learners will be able to...

- **Define pre-order, in-order, and post-order traversals**
- **Implement the three aforementioned traversals**
- **Analyze traversal applications**

Tree Traversal

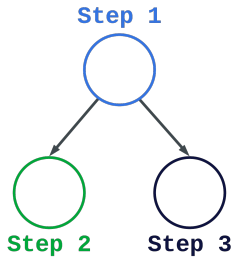
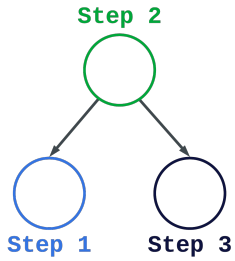
Iterating Over a Tree

Tree traversal is a foundational concept that involves visiting every node in a tree exactly once. As trees are a non-linear, hierarchical data structure, traversal is not as straightforward as it is with linear data structures. With arrays and linked lists, you can use a single loop can traverse the entire structure.

Tree traversal is crucial in various applications, such as executing searches and performing sorts. If we are successfully hitting all nodes, then we can perform operations on all of them. One operation we will focus on to show traversal is printing our nodes. We will delve deeply into tree traversal methodologies, focusing on implementations in C++.

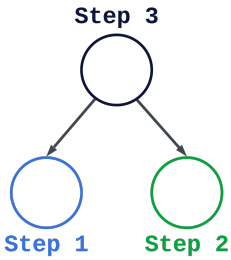
Traversal Strategies

In the context of trees, traversal pertains to the systematic visitation of all nodes in a specific order. There are several strategies to traverse a tree, each with its unique approach and utility. Here are the primary tree traversal strategies we will focus on:

Traversal Strategy	Graphical Representation	Time Complexity
Pre-Order • Step 1: Visit the root. • Step 2: Visit the left subtree. • Step 3: Visit the right subtree.		$O(n)$
In-Order • Step 1: Visit the left subtree. • Step 2: Visit the root. • Step 3: Visit the right subtree.		$O(n)$

Post-Order

- Step 1: Visit the left subtree.
- Step 2: Visit the right subtree.
- Step 3: Visit the root.



$O(n)$

The idea behind traversing a tree is to iterate over every node in the structure. Pre-order, in-order, and post-order traversals visit all of the nodes. They only differ on the order of traversal. Because of these, these three traversal strategies share the same time complexity, $O(n)$.

The traversal strategy depends largely on what you are looking to do. Pre-order traversal works best for making a copy of a tree or any other action that depends on the root first. In-order traversal is useful for when you want to sort a tree in ascending order, and post-order traversal is useful for when deleting nodes of a tree or any actions that depend on accessing the children before the root.

Creating Our Tree

Representing a Tree

To perform a traversal, we first need to represent a tree. We can represent a tree using nodes, where each node has a value, a pointer to its left child, and a pointer to its right child. Below is the C++ representation of the tree.

Node and Tree Classes

Let's start with the `TreeNode` class. It has attributes to store data and pointers to nodes on the left and right. We also need a constructor to set the initial values. Data to be stored will be passed to the constructor, while the left and right pointers should be set to `nullptr`.

```
class TreeNode {
public:
    int data;
    TreeNode* left;
    TreeNode* right;

    TreeNode(int data) {
        this->data = data;
        this->left = nullptr;
        this->right = nullptr;
    }
};
```

In our `Tree` class, we need a root node. Have the constructor set `root` to a `TreeNode` object.

```
class Tree {
public:
    TreeNode* root;

    Tree(int data) {
        root = new TreeNode(data);
    }
};
```

Adding Nodes

This is the basic tree structure we are familiar with, and we are going to replicate in code:

```
        5
      /   \
     3     8
    / \   / \
   1   4 7   9
```

Create a `main` method that instantiates a Tree object with 5 as the value to store. The `root's` left child has the value `3`, and the right child has the value `8`. The node with `3` (`tree.root->left`) has `1` as the left child and `4` as the right child. The `8` node (`tree.root->right`) has `7` as the left child and `9` as the right child.

```
int main() {
    // Creating the nodes for the tree
    Tree tree(5);
    tree.root->left = new TreeNode(3);
    tree.root->right = new TreeNode(8);
    tree.root->left->left = new TreeNode(1);
    tree.root->left->right = new TreeNode(4);
    tree.root->right->left = new TreeNode(7);
    tree.root->right->right = new TreeNode(9);

    // The tree is now created and can be traversed.
    return 0;
}
```

This code does not produce any output, but clicking the button below should not produce any errors.

▼ Code
Your code should look like this:

```cpp
class TreeNode {
public:
    int data;
    TreeNode* left;
    TreeNode* right;

    TreeNode(int data) {
        this->data = data;
        this->left = nullptr;
        this->right = nullptr;
    }
};

class Tree {
public:
    TreeNode* root;

    Tree(int data) {
        root = new TreeNode(data);
    }
};

int main() {
    // Creating the nodes for the tree
    Tree tree(5);
    tree.root->left = new TreeNode(3);
    tree.root->right = new TreeNode(8);
    tree.root->left->left = new TreeNode(1);
    tree.root->left->right = new TreeNode(4);
    tree.root->right->left = new TreeNode(7);
    tree.root->right->right = new TreeNode(9);

    // The tree is now created and can be traversed.
    return 0;
}
```

This page establishes an example tree structure, providing a tangible foundation for further traversal implementations. In the upcoming sections, we will delve into different tree traversal methods and implement them in C++. The tree on this page will serve as our reference example. Keep this tree structure in mind as we go through the various traversal methodologies and try to apply each method to this example to enhance your understanding of tree traversal in C++

Pre-Order Traversal

Traversing the Root Node First

Pre-order traversal is one of the most common methods to traverse a tree. In this traversal method, you visit the root node first, then recursively do a pre-order traversal of the left subtree, followed by a recursive pre-order traversal of the right subtree.

Remember, our tree looks like this:

```
       5
     /   \
    3     8
   / \   / \
  1   4 7   9
```

That means if we conduct a pre-order traversal, we would iterate through the data structure in this order:

```
5 -> 3 -> 1 -> 4 -> 8 -> 7 -> 9
```

Implementation

Trees are a self-similar data structure. That means the overall structure of the entire tree can be found in each of the subtrees. These types of data structures lend themselves to recursion, which is how we are going to perform each of these traversals.

Start by creating the `preOrderTraversal` method inside the `Tree` class. It takes a `TreeNode` object as a parameter. The base case is when the given TreeNode object is `nullptr`. Otherwise, print the data stored in the given node. The recurse on the left node. Finally recurse on the right node.

```
    void preOrderTraversal(TreeNode* node) {
        if (node != nullptr) {
            // Visit the root node
            std::cout << node->data << " ";
            // Traverse the left subtree in pre-order
            preOrderTraversal(node->left);
            // Traverse the right subtree in pre-order
            preOrderTraversal(node->right);
        }
    }
```

Now think about how we would call this recursive method. Because it expects a `TreeNode` object as a parameter, we would have to call it like this (note, this is just an example, do not copy/paste it into the IDE):

```
tree.preOrderTraversal(tree.root);
```

This seems a bit clumsy. The root node is the first node in a tree. Should we not automatically start there? To get around this issue, we are going to perform some method overloading. Declare `preOrderTraversal` again, but this time there are no parameters. In the body, call the `preOrderTraversal` method and pass it the root attribute. This way, we can say tree.`preOrderTraversal()` and traverse the entire tree.

```
    void preOrderTraversal() {
        preOrderTraversal(root);
        std::cout << std::endl;
    }
```

Update the `main` method to include a call to the `preOrderTraversal` method. You do not need to alter any of the code that creates our tree.

```
int main() {
    // Creating the nodes for the tree
    Tree tree(5);
    tree.root->left = new TreeNode(3);
    tree.root->right = new TreeNode(8);
    tree.root->left->left = new TreeNode(1);
    tree.root->left->right = new TreeNode(4);
    tree.root->right->left = new TreeNode(7);
    tree.root->right->right = new TreeNode(9);

    // Perform and print the result of in-order traversal
    std::cout << "Pre-Order Traversal:" << std::endl;
    tree.preOrderTraversal();

    return 0;
}
```

▼ Code

Your code should look like this:

```
class TreeNode {
public:
    int data;
    TreeNode* left;
    TreeNode* right;

    TreeNode(int data) {
        this->data = data;
        this->left = nullptr;
        this->right = nullptr;
```

```
    }
};

class Tree {
public:
    TreeNode* root;

    Tree(int data) {
        root = new TreeNode(data);
    }

    void preOrderTraversal(TreeNode* root) {
        if (root != nullptr) {
            // Visit the root node
            std::cout << root->data << " ";
            // Traverse the left subtree in pre-order
            preOrderTraversal(root->left);
            // Traverse the right subtree in pre-order
            preOrderTraversal(root->right);
        }
    }

    void preOrderTraversal() {
        preOrderTraversal(root);
        std::cout << std::endl;
    }

};

int main() {
    // Creating the nodes for the tree
    Tree tree(5);
    tree.root->left = new TreeNode(3);
    tree.root->right = new TreeNode(8);
    tree.root->left->left = new TreeNode(1);
    tree.root->left->right = new TreeNode(4);
    tree.root->right->left = new TreeNode(7);
    tree.root->right->right = new TreeNode(9);

    // Perform and print the result of pre-order traversal
    std::cout << "Pre-Order Traversal:" << std::endl;
    tree.preOrderTraversal();

    return 0;

}
```

Running the program should produce the following output:

```
Pre-order Traversal:
5 3 1 4 8 7 9
```

This output is consistent with pre-order traversal – visiting the root node first, then traversing the left subtree, followed by the right subtree. After mastering this traversal technique, understanding other traversal methods becomes more intuitive and manageable.

In-Order Traversal

Traversing the Left Node First

In-order Traversal is another essential tree traversal method, and it is especially useful for binary search trees. In this traversal method, you recursively visit the left subtree, visit the root node, and finally recursively visit the right subtree.

Our tree looks like this:

```
      5
    /   \
   3     8
  / \   / \
 1   4 7   9
```

That means if we conduct an in-order traversal, we would iterate through the data structure in this order:

```
1 -> 3 -> 4 -> 5 -> 7 -> 8 -> 9
```

Implementation

Start by creating the `inOrderTraversal` method in the `Tree` class. The method takes a `TreeNode` object as a parameter. As long as this node is not `nullptr`, recursively call `inOrderTraversal` with the left subtree (`node->left`). When this is complete, visit the node itself. Lastly, recursively visit the right subtree (`node->right`).

```
    void inOrderTraversal(TreeNode* node) {
        if (node != nullptr) {
            // Traverse the left subtree in in-order
            inOrderTraversal(node->left);
            // Visit the root node
            std::cout << node->data << " ";
            // Traverse the right subtree in in-order
            inOrderTraversal(node->right);
        }
    }
```

Again, we are going to overload the `inOrderTraversal` method so that we can call it without any arguments and have the method start with the root node.

```
    void inOrderTraversal() {
        inOrderTraversal(root);
        std::cout << std::endl;
    }
```

Update the `main` method to include a call to the `inOrderTraversal` method. You do not need to alter any of the code that creates our tree.

```
int main() {
    // Creating the nodes for the tree
    Tree tree(5);
    tree.root->left = new TreeNode(3);
    tree.root->right = new TreeNode(8);
    tree.root->left->left = new TreeNode(1);
    tree.root->left->right = new TreeNode(4);
    tree.root->right->left = new TreeNode(7);
    tree.root->right->right = new TreeNode(9);

    // Perform and print the result of in-order traversal
    std::cout << "In-Order Traversal:" << std::endl;
    tree.inOrderTraversal();

    return 0;
}
```

▼ Code

Your code should look like this:

```


class TreeNode {
public:
    int data;
    TreeNode* left;
    TreeNode* right;

    TreeNode(int data) {
        this->data = data;
        this->left = nullptr;
        this->right = nullptr;
    }
};

class Tree {
public:
    TreeNode* root;

    Tree(int data) {
        root = new TreeNode(data);
    }

    void preOrderTraversal(TreeNode* root) {
        if (root != nullptr) {
            // Visit the root node
```

```
            std::cout << root->data << " ";
            // Traverse the left subtree in pre-order
            preOrderTraversal(root->left);
            // Traverse the right subtree in pre-order
            preOrderTraversal(root->right);
        }
    }

    void preOrderTraversal() {
        preOrderTraversal(root);
        std::cout << std::endl;
    }

    void inOrderTraversal(TreeNode* node) {
        if (node != nullptr) {
            // Traverse the left subtree in in-order
            inOrderTraversal(node->left);
            // Visit the root node
            std::cout << node->data << " ";
            // Traverse the right subtree in in-order
            inOrderTraversal(node->right);
        }
    }

    void inOrderTraversal() {
        inOrderTraversal(root);
        std::cout << std::endl;
    }
};

int main() {
    // Creating the nodes for the tree
    Tree tree(5);
    tree.root->left = new TreeNode(3);
    tree.root->right = new TreeNode(8);
    tree.root->left->left = new TreeNode(1);
    tree.root->left->right = new TreeNode(4);
    tree.root->right->left = new TreeNode(7);
    tree.root->right->right = new TreeNode(9);

    // Perform and print the result of in-order traversal
    std::cout << "In-Order Traversal:" << std::endl;
    tree.inOrderTraversal();

    return 0;
}
```

Running the program should produce the following output:

```
In-Order Traversal:
1 3 4 5 7 8 9
```

In-order traversal is particularly useful in binary search trees, where it returns the nodes in ascending order (we will cover binary search trees in greater detail in a later assignment). Mastering in-order traversal is crucial for efficient manipulation and utilization of tree structures. The next step is to understand post-order traversal.

Post-Order Traversal

Traversing the Root Node Last

Post-order traversal is a method of tree traversal in which you recursively visit the left subtree, recursively visit the right subtree, and then visit the root node.

Our tree looks like this:

```
      5
    /   \
   3     8
  / \   / \
 1   4 7   9
```

That means if we conduct a post-order traversal, we would iterate through the data structure in this order:

```
1 -> 4 -> 3 -> 7 -> 9 -> 8 -> 5
```

Implementation

Start by creating the `postOrderTraversal` method in the `Tree` class. The method takes a `TreeNode*` as a parameter. As long as this node is not `nullptr`, recursively call `postOrderTraversal` with the left subtree (`node->left`). When this is complete, recursively visit the right subtree (`node->right`). Lastly, visit the node itself.

```
    void postOrderTraversal(TreeNode* node) {
        if (node != nullptr) {
            // Traverse the left subtree in post-order
            postOrderTraversal(node->left);
            // Traverse the right subtree in post-order
            postOrderTraversal(node->right);
            // Visit the root node
            std::cout << node->data << " ";
        }
    }
```

Again, we are going to overload the `postOrderTraversal` method so that we can call it without any arguments and have the method start with the root node.

```
    void postOrderTraversal() {
        postOrderTraversal(root);
        std::cout << std::endl;
    }
```

Update the main method to include a call to the `postOrderTraversal` method. You do not need to alter any of the code that creates our tree.

```
int main() {
    // Creating the nodes for the tree
    Tree tree(5);
    tree.root->left = new TreeNode(3);
    tree.root->right = new TreeNode(8);
    tree.root->left->left = new TreeNode(1);
    tree.root->left->right = new TreeNode(4);
    tree.root->right->left = new TreeNode(7);
    tree.root->right->right = new TreeNode(9);

    // Perform and print the result of post-order traversal
    std::cout << "Post-Order Traversal:" << std::endl;
    tree.postOrderTraversal();

    return 0;
}
```

▼ Code

Your code should look like this:

```
class TreeNode {
public:
    int data;
    TreeNode* left;
    TreeNode* right;

    TreeNode(int data) {
        this->data = data;
        this->left = nullptr;
        this->right = nullptr;
    }
};

class Tree {
public:
    TreeNode* root;

    Tree(int data) {
        root = new TreeNode(data);
    }

    void preOrderTraversal(TreeNode* root) {
        if (root != nullptr) {
            // Visit the root node
            std::cout << root->data << " ";
```

```
            // Traverse the left subtree in pre-order
            preOrderTraversal(root->left);
            // Traverse the right subtree in pre-order
            preOrderTraversal(root->right);
        }
    }

    void preOrderTraversal() {
        preOrderTraversal(root);
        std::cout << std::endl;
    }

    void inOrderTraversal(TreeNode* node) {
        if (node != nullptr) {
            // Traverse the left subtree in in-order
            inOrderTraversal(node->left);
            // Visit the root node
            std::cout << node->data << " ";
            // Traverse the right subtree in in-order
            inOrderTraversal(node->right);
        }
    }

    void inOrderTraversal() {
        inOrderTraversal(root);
        std::cout << std::endl;
    }

    void postOrderTraversal(TreeNode* node) {
        if (node != nullptr) {
            // Traverse the left subtree in post-order
            postOrderTraversal(node->left);
            // Traverse the right subtree in post-order
            postOrderTraversal(node->right);
            // Visit the root node
            std::cout << node->data << " ";
        }
    }

    void postOrderTraversal() {
        postOrderTraversal(root);
        std::cout << std::endl;
    }
};

int main() {
    // Creating the nodes for the tree
    Tree tree(5);
    tree.root->left = new TreeNode(3);
    tree.root->right = new TreeNode(8);
    tree.root->left->left = new TreeNode(1);
    tree.root->left->right = new TreeNode(4);
    tree.root->right->left = new TreeNode(7);
    tree.root->right->right = new TreeNode(9);

    // Perform and print the result of post-order traversal
    std::cout << "Post-Order Traversal:" << std::endl;
```

```
    tree.postOrderTraversal();

    return 0;
}
```

Running the program should produce the following output:

```
Post-Order Traversal:
1 4 3 7 9 8 5
```

Post-Order Traversal is indispensable in scenarios where you need to perform operations on a node only after performing equivalent operations on its descendants, for example, while deleting a tree.

In the following assignments, we will delve into more complex tree manipulation techniques and explore additional tree traversal methodologies that can enhance your tree manipulation and analysis skills.

Formative Assessment 1

Formative Assessment 2